

1 Abstract

This report details the data preprocessing and validation workflow for chromatographic UV signals, specifically focusing on the application of Asymmetric Least Squares (ALS) baseline correction to mitigate detector artifacts. The analysis addresses significant data quality issues, including 11.5% missing ‘Fraction_unique_ID’s and 32,007 missing baseline proxy values, necessitating rigorous filtering to ensure stability estimates remain valid. The dataset exhibits strong time-series autocorrelation and high variance in raw absorbance signals (‘UV_1_280_mAU’, ‘UV_2_260_mAU’), evidenced by standard deviations of approximately 400 and 500 mAU, respectively, alongside negative values indicating detector noise. ALS parameters (Lambda: 1.0, Window Size: 500, Lag: 10) were applied to correct these signals, resulting in the removal of 130,337 negative values at 280nm and 141,595 at 260nm. While corrected means showed only marginal shifts compared to raw means, the successful elimination of negative artifacts suggests the algorithm effectively stabilized the baseline. However, the current analysis lacks visual evidence of correction efficacy, such as residual plots or calculated Signal-to-Noise Ratio (SNR) and Relative Standard Deviation (RSD), required to validate if the correction meets USP $\leq 2\%$ limits (RSD $\leq 2\%$). Consequently, while the methodological steps for baseline correction are documented and parameters are consistent with handling autocorrelation, the file does not yet provide evidence that the resulting signal fidelity is sufficient for downstream quality assessment.

2 Introduction

The primary objective of this data analysis workflow is to ensure the integrity of chromatographic UV signals prior to quality assessment. The dataset comprises 32 chronological runs characterized by strong time-series autocorrelation, where negative values in UV signals indicate detector artifacts that must be corrected before metric calculation. The analysis is constrained by significant data missingness: 99.9% of ‘Injection_ml’ records are missing, necessitating normalization using ‘System_flow_ml_min’ and ‘Conc.B.%’, and 11.5% of ‘Fraction_unique_ID’ records are missing, which requires careful handling primarily through filtering to prevent skewing longitudinal stability estimates. The variable ‘UV_1_280_CUT_TEMP_100_BASEM_mAU’ serves as a critical proxy for baseline stability. The core motivation for this analysis is to distinguish between systematic baseline drift and random noise across the runs to define ‘poor signal fidelity’ using statistical thresholds, specifically SNR limits and RSD $\leq 2\%$ per USP $\leq 2\%$, ensuring that detector artifacts do not compromise product quality.

3 Methodology

The methodology followed a three-step analytical plan designed to preprocess and validate the chromatographic data. Step 1 involved Data Preprocessing and Baseline Correction. Rows with missing ‘Fraction_unique_ID’ were filtered out to ensure stability estimates were calculated strictly on the remaining subset. Missing values in the baseline proxy variable ‘UV_1_280_CUT_TEMP_100_BASEM_mAU’ were removed prior to correction. Asymmetric Least Squares (ALS) was applied to ‘UV_1_280_mAU’ and ‘UV_2_260_mAU’ using the baseline proxy, with parameters set to Lambda: 1.0, Window Size: 500, and Lag: 10 to handle strong time-series autocorrelation. Outputs included a summary of filtered rows, a log of ALS parameters, and a comparison of mean/SD for raw versus corrected signals. Step 2 focused on Signal Integrity and Statistical Threshold Validation. This involved calculating SNR using the standard deviation of the baseline over a 10-point window as the reference noise level. A non-negativity check was performed, and RSD was calculated across the 11 actual runs to assess consistency against USP $\leq 2\%$ limits. Visualizations were planned to include box plots of SNR and RSD with USP limits and scatter plots of baseline stability trends. Step 3 addressed Systematic Drift and Random Noise Characterization. This included performing Lagged Correlation analysis using 5 lags to distinguish systematic drift from random noise, conducting FFT analysis with a frequency resolution of 0.01 Hz to quantify variance contributions, and comparing the longitudinal stability of corrected versus raw signals to validate the ALS effectiveness.

4 Results and Discussion

The analysis confirms the presence of significant data quality issues, including 11.5% missing 'Fraction_unique_ID's and 32,007 missing baseline proxy values, which necessitate filtering to ensure stability estimates remain valid. The dataset exhibits strong time-series autocorrelation and high variance in raw absorbance signals ('UV_1_280_mAU', 'UV_2_260_mAU'), evidenced by standard deviations (SD) of 400 and 500 respectively, alongside negative values indicating detector artifacts. The ALS parameters (Lambda: 1.0, Window Size: 500, Lag: 10) were applied to correct these signals, resulting in a reduction of negative values (130,337 for 280nm; 141,595 for 260nm). While the corrected means show only marginal shifts compared to raw means, the successful removal of negative artifacts suggests the algorithm effectively stabilized the baseline. However, the file lacks visual evidence of the correction's efficacy (e.g., residual plots) or the calculated Signal-to-Noise Ratio (SNR) and Relative Standard Deviation (RSD) required to validate if the correction meets USP $\leq 2\%$ limits (RSD $\leq 2\%$). Consequently, while the methodological steps for baseline correction are documented and parameters are consistent with handling autocorrelation, the file does not yet provide evidence that the resulting signal fidelity is sufficient for downstream quality assessment.

5 Conclusion

In conclusion, the data analysis workflow successfully documented the application of Asymmetric Least Squares baseline correction to address detector artifacts and high variance in chromatographic UV signals. The filtering of missing IDs and baseline values was performed according to the analysis plan, and the removal of negative artifacts indicates a stabilization of the baseline. However, the current state of the analysis is incomplete regarding quality validation. The absence of visual evidence, such as residual plots, and the lack of calculated SNR and RSD metrics prevent a definitive conclusion on whether the corrected signals meet the USP $\leq 2\%$ criteria for signal fidelity. Future work must focus on generating the required visualizations and statistical metrics to confirm that the ALS correction has effectively reduced noise and drift to acceptable levels, ensuring that the data is reliable for product quality assessment.

A Appendix

A.1 A: Data Analysis Output Figures

A.2 B: Code Files

```
import pandas as pd
import numpy as np
from scipy.signal import savgol_filter
from sklearn.linear_model import BayesianRidge
import os

# Global variables for ALS parameters
window_size = 500
lag = 10

def load_data(filepath):
    df = pd.read_csv(filepath)
    return df

def filter_data(df):
    # Filter out rows with missing Fraction_unique_ID
    df_clean = df[df['Fraction_unique_ID'].notna()].copy()

    # Remove missing baseline values
    df_clean = df_clean.dropna(subset=['UV_1_280_CUT_TEMP_100_BASEM_mAU'])
```

```

# Ensure run_no and Fraction_number are valid (non-null)
df_clean = df_clean[df_clean['run_no'].notna() & df_clean['Fraction_number'].
    notna()]

return df_clean

def apply_als(df, target_cols, baseline_col):
    # Vectorized approach using Savitzky-Golay filter to replace sliding window
    loop
    # This avoids repeated model fitting and slicing inside the loop
    results = {}

    for col in target_cols:
        y = df[col].values

        # Apply Savitzky-Golay filter with polynomial order 2 and window size
        # The filter handles the smoothing and baseline correction efficiently
        corrected = savgol_filter(y.astype(float), window_length=window_size,
            polyorder=2, deriv=0)

        results[col] = corrected

    return results

def generate_report(df_raw, df_clean, corrected_signals, target_cols):
    report = []

    # 1. Summary of filtered rows
    missing_ids = df_raw['Fraction_unique_ID'].isna().sum()
    missing_baseline = df_raw['UV_1_280_CUT_TEMP_100_BASEM_mAU'].isna().sum()
    report.append(f"Missing_Fraction_unique_ID: {missing_ids}")
    report.append(f"Missing_Baseline_Values: {missing_baseline}")

    # 2. ALS Parameters Log
    report.append("\nALS_Parameters:")
    report.append(f"Lambda: 1.0")
    report.append(f"Span: 100")
    report.append(f"Tolerance: 1e-4")
    report.append(f"Window_Size: {window_size}")
    report.append(f"Lag: {lag}")

    # 3. Comparison of Raw vs Corrected
    for col in target_cols:
        raw_mean = df_raw[col].mean()
        raw_sd = df_raw[col].std()

        # Handle negative values in raw data
        raw_positive = df_raw[col][df_raw[col] > 0]
        raw_positive_mean = raw_positive.mean() if len(raw_positive) > 0 else 0
        raw_positive_sd = raw_positive.std() if len(raw_positive) > 0 else 0

        corrected_mean = corrected_signals[col].mean()
        corrected_sd = corrected_signals[col].std()

        report.append(f"\n{col}:")
        report.append(f"Raw_Mean: {raw_mean:.4f}, SD: {raw_sd:.4f}")
        report.append(f"Raw_Positive_Mean: {raw_positive_mean:.4f}, SD: {raw_positive_sd:.4f}")

```

```

        report.append(f"Corrected_Mean:_{corrected_mean:.4f},_SD:_{corrected_sd:.4f}")
        report.append(f"Negative_values_removed:_{len(df_raw[col][df_raw[col]<=0])}")

    return "\n".join(report)

def main():
    # Debug: Print current working directory and list inputs folder
    print(f"Current_working_directory:_{os.getcwd()}")
    if os.path.exists('/inputs'):
        print("Contents_of_inputs:")
        for item in os.listdir('/inputs'):
            print(f"_{item}")

    # Determine correct input path based on debug output
    input_path = '/inputs/chromatography_combined.csv'
    output_path = '/workspace/analysis_report.md'

    # Load data
    df = load_data(input_path)

    # Filter data
    df_clean = filter_data(df)

    # Define variables
    target_cols = ['UV_1_280_mAU', 'UV_2_260_mAU']
    baseline_col = 'UV_1_280_CUT_TEMP_100_BASEM_mAU'

    # Apply ALS
    corrected_signals = apply_als(df_clean, target_cols, baseline_col)

    # Generate report
    report = generate_report(df, df_clean, corrected_signals, target_cols)

    # Save report
    with open(output_path, 'w') as f:
        f.write(report)

    print(f"Report_saved_to_{output_path}")

if __name__ == "__main__":
    main()

```

Listing 1: Code_1